CLOUD 9 SYSTEMS

INFORME TÉCNICO

Pruebas funcionales y revisión de seguridad

Alcance	Pruebas T01–T10 y revisión de controles mínimos de seguridad.
Entorno	Laravel · PHP 8.5.4 · PHPUnit · SQLite en memoria.
Resultado comunicado	18 pruebas · 88 aserciones · 0 fallos
Alcance de cambios	Se añadieron pruebas en tests/. No se modificó código de producción ni configuración del proyecto.

Documento de apoyo a la entrega académica

La documentación describe el comportamiento comprobado y señala expresamente los casos no aplicables o las limitaciones de las pruebas.

ÍNDICE

1. Introducción 3

2. Entorno de pruebas 3

3. Resultados de las pruebas funcionales 3

3.1. T01 — Presupuesto con pago íntegro e importe exacto . . . 3

3.2. T02 — Pago fraccionado y validación de precio . . . 3

3.3. T03 — Regla de bloqueo . . . 3

3.4. T04 — Aviso de confirmación . . . 4

3.5. T05 — Alta válida . . . 4

3.6. T06 — Alta inválida . . . 4

3.7. T07 — CRUD y relación Eloquent . . . 4

3.8. T08 — Subida o sustitución de archivos . . . 4

3.9. T09 — Inmutabilidad del presupuesto histórico . . . 4

3.10. T10 — Integridad de transacciones y rollback . . . 4

4. Revisión de seguridad mínima 5

4.1. Protección CSRF . . . 5

4.2. Validación en servidor . . . 5

4.3. Escape de HTML . . . 5

4.4. Control de asignación masiva . . . 5

4.5. Integridad de relaciones . . . 5

4.6. Secretos y archivos de entrega . . . 5

5. Resumen de resultados 6

6. Archivos de pruebas creados 6

7. Alcance de los cambios y reproducibilidad 7

8. Conclusiones 7

1. Introducción

Este informe recoge las pruebas funcionales y la revisión de seguridad realizadas sobre Cloud 9 Systems. El objetivo ha sido comprobar, mediante pruebas automatizadas, los comportamientos que existen realmente en la aplicación y documentar con claridad las adaptaciones y limitaciones.

El Plan Maestro y la implementación existente han sido la referencia. No se han incorporado funcionalidades ficticias ni se ha modificado el código de producción para hacer encajar los requisitos académicos.

Criterio de trabajo: probar lo que la aplicación implementa; adaptar la comprobación cuando el requisito no coincide exactamente con el diseño; declarar como no aplicable aquello que no existe.

2. Entorno de pruebas

- Framework: Laravel.
- PHP: 8.5.4 en CLI, Artisan y PHPUnit.
- Pruebas: PHPUnit mediante la configuración existente del proyecto.
- Base de datos de pruebas: SQLite en memoria, con RefreshDatabase.
- Extensiones necesarias: pdo_sqlite y sqlite3.
- Aislamiento: las pruebas no utilizaron ni modificaron la base de datos MySQL de desarrollo.

Las pruebas se ejecutan sobre una base de datos temporal. Los datos de ejemplo se preparan para cada caso y no deben confundirse con datos reales de producción o desarrollo.

3. Resultados de las pruebas funcionales

Resultado comunicado por Cursor tras la ejecución: 18 tests, 88 assertions y 0 failures. Los casos T01–T07, T09 y T10 se cubrieron con pruebas automatizadas. T08 se declaró no aplicable porque la aplicación no contiene actualmente un flujo de subida de archivos.

3.1. T01 — Presupuesto con pago íntegro e importe exacto

Objetivo: comprobar el cálculo del importe de un presupuesto cuando se utiliza la modalidad de pago íntegro.

Comprobación: se prepararon seis productos con precios 10, 20, 30, 40, 50 y 60. El total esperado y obtenido fue 210, con los seis productos asociados al presupuesto.

Resultado: PASS. Se valida la suma exacta de los importes existentes en el sistema. No se ha supuesto una fórmula de precios adicional.

3.2. T02 — Pago fraccionado y validación de precio

Objetivo: comprobar la modalidad fraccionada disponible y una validación de límite de precio.

Comprobación: la modalidad Fraccionado es aceptada, pero el importe total se calcula con la misma suma de productos que en el pago íntegro. Además, un precio negativo (-5) es rechazado.

Resultado: PASS con adaptación. La aplicación no implementa un calendario ni un cálculo de cuotas; la prueba no afirma que exista esa funcionalidad.

3.3. T03 — Regla de bloqueo

Objetivo: comprobar que una operación no permitida queda bloqueada.

Comprobación: se intenta editar o eliminar un producto protegido. La operación queda impedida y el producto permanece en la base de datos.

Resultado: PASS. Se verifica la regla de protección que existe en la aplicación.

3.4. T04 — Aviso de confirmación

Objetivo: comprobar la presencia del mecanismo de confirmación antes de abandonar una operación o navegación protegida.

Comprobación: la respuesta HTML contiene los elementos del mecanismo confirm-nav y la llamada confirm() utilizada por el JavaScript.

Resultado: PASS con alcance limitado. La prueba automatizada comprueba el marcado y el código servido; no ejecuta un navegador real ni demuestra la interacción visual del diálogo.

3.5. T05 — Alta válida

Objetivo: comprobar que un alta válida se guarda correctamente.

Comprobación: se crea un producto válido de AWS, región Ireland, categoría Storage y precio 25. Se comprueba su persistencia y que el campo protegido queda en false.

Resultado: PASS.

3.6. T06 — Alta inválida

Objetivo: comprobar que los datos incompatibles o inválidos son rechazados y no se guardan.

Comprobación: se envía una combinación incompatible de proveedor AWS con región West Europe (asociada a Azure), además de un precio decimal 12.5 que no cumple la validación configurada. No se crean registros inválidos.

Resultado: PASS. La prueba verifica validaciones existentes, no reglas añadidas exclusivamente para el test.

3.7. T07 — CRUD y relación Eloquent

Objetivo: comprobar una operación de persistencia y una relación entre entidades.

Comprobación: se crea un proyecto con dos requisitos y se consulta la relación Eloquent, que devuelve los dos registros asociados.

Resultado: PASS.

3.8. T08 — Subida o sustitución de archivos

Objetivo académico: comprobar subida, sustitución o rechazo de archivos mediante Storage::fake y UploadedFile::fake.

Resultado: NO APLICABLE. La implementación auditada no contiene campos de subida de archivos, flujo de almacenamiento ni comportamiento de dominio que permita realizar esta prueba de forma real. No se ha creado un test ficticio ni se ha añadido una funcionalidad nueva para satisfacer el requisito.

3.9. T09 — Inmutabilidad del presupuesto histórico

Objetivo: comprobar que un presupuesto ya creado conserva su importe aunque cambie posteriormente el catálogo.

Comprobación: se crea un presupuesto con importe 210 y, después, se modifica directamente el precio del producto en la base de datos de pruebas a 9999. El presupuesto conserva el importe histórico de 210.

Resultado: PASS. La comprobación se limita al importe histórico almacenado en el presupuesto. No se afirma que el total de una arquitectura sea histórico, ya que ese cálculo depende del catálogo actual.

3.10. T10 — Integridad de transacciones y rollback

Objetivo: comprobar que una operación fallida no deja datos parciales.

Comprobación: se provoca una excepción durante la creación de un presupuesto, después de iniciarse la operación de persistencia. La petición devuelve HTTP 500 y se verifica que la transacción revierte los

registros del presupuesto y sus asociaciones.

Resultado: PASS. La prueba verifica el rollback y la ausencia de datos incompletos.

4. Revisión de seguridad mínima

4.1. Protección CSRF

Se revisaron los mecanismos presentes: token CSRF en el formulario, meta csrf-token, uso del token en JavaScript y middleware de protección de peticiones. En el entorno de PHPUnit, Laravel desactiva el comportamiento habitual de rechazo CSRF durante runningUnitTests(); por ello, no se presenta una respuesta HTTP 419 como prueba automatizada superada.

Resultado: mecanismo revisado en el código. No se afirma haber probado el rechazo 419 mediante PHPUnit.

4.2. Validación en servidor

Se comprobaron las reglas de validación de las peticiones y el rechazo de datos inválidos. Las pruebas T05 y T06 verifican tanto la aceptación de datos correctos como la ausencia de persistencia de datos rechazados.

Resultado: PASS en los escenarios cubiertos.

4.3. Escape de HTML

En las vistas Blade revisadas se utiliza la salida escapada habitual {{ }} y no se localizaron usos de {!! ... !!} en el conjunto inspeccionado. También se identificó la función JavaScript escapeHtml() para escapar contenido.

Resultado: revisión satisfactoria del código inspeccionado. No equivale a una auditoría exhaustiva de todos los posibles vectores de XSS.

4.4. Control de asignación masiva

Se envió un valor protegido manipulado (protegido=1) durante un alta. La petición no consiguió establecer el producto como protegido: el registro persistido quedó con protegido=false, conforme al flujo de alta revisado.

Resultado: PASS para el escenario probado.

4.5. Integridad de relaciones

Se comprobó que un producto asociado/protegido no puede eliminarse mediante el flujo bloqueado y que las relaciones probadas se mantienen correctamente.

Resultado: PASS en los escenarios cubiertos.

4.6. Secretos y archivos de entrega

La revisión identificó que .env está incluido en .gitignore. El archivo .env existe en el entorno local y debe permanecer fuera de cualquier ZIP o entrega. En el momento de la auditoría no se disponía de un directorio .git en el espacio de trabajo revisado, por lo que no se certificó el contenido de un commit.

Acción obligatoria antes de entregar: comprobar el contenido real del ZIP y del repositorio remoto para confirmar que no se han incluido .env, credenciales, claves ni otros secretos.

5. Resumen de resultados

Prueba	Resultado	Descripción
T01	PASS	Total exacto del presupuesto con pago íntegro.
T02	PASS*	Pago fraccionado aceptado y precio negativo rechazado; sin cálculo de cuotas.
T03	PASS	Bloqueo de edición/eliminación de producto protegido.
T04	PASS*	Se comprueba el mecanismo en HTML/JS; no se ejecuta el diálogo en navegador.
T05	PASS	Alta válida persistida.
T06	PASS	Alta inválida rechazada y no persistida.
T07	PASS	Proyecto y relación con dos requisitos.
T08	N/A	No existe funcionalidad de subida de archivos.
T09	PASS	El importe del presupuesto conserva su valor histórico.
T10	PASS	Rollback tras excepción, sin datos parciales.

* PASS con adaptación o alcance limitado, explicado en el apartado correspondiente.

Prueba	Resultado	Descripción
Seguridad	Resultado	Alcance
CSRF	Revisado	Mecanismos presentes; no se fuerza HTTP 419 en PHPUnit.
Validación	PASS	Validación en servidor comprobada en escenarios cubiertos.
Escape HTML	Revisado	Vistas y JavaScript inspeccionados.
Mass assignment	PASS	El intento de modificar protegido no altera el valor.
Relaciones	PASS	Integridad comprobada en los casos cubiertos.
Secretos	Pendiente final	Verificar ZIP y Git antes de publicar.

6. Archivos de pruebas creados

Las pruebas y el soporte común se encuentran en tests/. Los archivos comunicados como creados son:

- tests/Support/PreparesCloud9Data.php
- tests/Feature/T01PresupuestoPagoIntegroImporteExactoTest.php
- tests/Feature/T02PresupuestoFraccionadoOLimitePrecioTest.php
- tests/Feature/T03ReglaBloqueoTest.php
- tests/Feature/T04AvisoConfirmacionTest.php
- tests/Feature/T05AltaValidaTest.php
- tests/Feature/T06AltaInvalidaTest.php
- tests/Feature/T07CrudORelacionEloquentTest.php
- tests/Feature/T09PresupuestoHistoricoInmutableTest.php

- tests/Feature/T10TransaccionRollbackTest.php
- tests/Feature/SeguridadMinimaTest.php

7. Alcance de los cambios y reproducibilidad

Según el informe de ejecución de Cursor, los cambios se limitaron a la incorporación de pruebas y soporte bajo tests/. No se modificaron app/, routes/, resources/, database/ ni phpunit.xml. La ejecución se realizó con SQLite en memoria, separada de la base de datos MySQL de desarrollo.

Para reproducir la ejecución, desde la raíz del proyecto y con las dependencias instaladas, se puede ejecutar:

```
php artisan test
```

La salida que se debe conservar como evidencia es la salida real de PHPUnit correspondiente a la versión final de los archivos incluidos en GitHub.

8. Conclusiones

Las pruebas automatizadas cubren los comportamientos compatibles con la implementación actual de Cloud 9 Systems: cálculo del presupuesto, validaciones, bloqueos, altas, relaciones, conservación del importe histórico y rollback transaccional.

Los requisitos no implementados o no ejecutables en el entorno de pruebas se han identificado expresamente. En particular, T08 no aplica porque no existe gestión de archivos; T04 verifica el mecanismo de confirmación sin simular un navegador; y CSRF se documenta como revisión del mecanismo, no como una prueba de rechazo 419.

El resultado comunicado es de 18 pruebas, 88 aserciones y 0 fallos. Antes de publicar la entrega, debe comprobarse que el repositorio contiene los archivos de pruebas y documentación correctos y que no se ha incluido ningún secreto, especialmente el archivo .env.

— Fin del informe —